

Assignment – 7

Implement and analyze various probability distributions in Python, including Binomial, Poisson, and Normal distributions.

✓ THEORY

1. What is the difference between Binomial, Poisson, and Normal distributions?

While all three are used to model the probability of events, they differ based on the nature of the data and the conditions of the experiment.

- **Binomial Distribution:** This is a **discrete** distribution used for a fixed number of independent trials (n), where each trial has only two possible outcomes: "success" or "failure." It is defined by the number of trials and the probability of success (p) in each trial.
 - *Example:* Calculating the probability of getting exactly 7 heads in 10 coin tosses.
 - **Poisson Distribution:** This is also a **discrete** distribution, but it focuses on the **count of events** occurring within a specific interval of time or space. Unlike the Binomial distribution, there is no fixed number of trials; instead, we use a rate (λ) representing the average number of occurrences.
 - *Example:* Predicting how many customers will enter a store between 10:00 AM and 11:00 AM.
 - **Normal (Gaussian) Distribution:** This is a **continuous** distribution that describes data that clusters around a central mean. It is defined by the mean (μ) and standard deviation (σ). It is perfectly symmetric, creating the famous "bell curve" shape where the mean, median, and mode are all equal.
 - *Example:* The distribution of test scores or human heights in a large population.
-

2. What is the real-world significance of the Poisson distribution?

The Poisson distribution is essential for modeling independent, random events that occur at a constant average rate. It is particularly significant in fields where "arrivals"

or "occurrences" need to be managed:

- **Network Reliability:** Estimating the probability of a specific number of packet losses or server requests in a given second to optimize bandwidth.
- **Public Health:** Modeling the spread of rare diseases or the frequency of emergency room visits to allocate medical resources.
- **Finance and Insurance:** Predicting the number of insurance claims filed in a month or the frequency of "black swan" market events.

3. What is the Central Limit Theorem and how does it relate to Normal distribution?

The **Central Limit Theorem (CLT)** is the foundation of frequentist statistics. It states that if you take a sufficiently large sample size from a population (usually $n \geq 30$), the distribution of the **sample means** will be approximately normal, regardless of the population's original distribution.

Relation to Normal Distribution: The CLT allows us to apply "Normal" logic to "Non-Normal" data. Because the sample means converge into a bell curve, we can use tools like Z-tests and confidence intervals to make accurate predictions about a population even if the underlying data is skewed or uniform.

4. Why is Normal distribution widely used in machine learning and statistics?

The Normal distribution is favored in data science because it simplifies complex systems into manageable mathematical models:

- **Error Modeling:** According to the Gauss-Markov theorem, in many linear models, the "errors" or residuals are assumed to follow a normal distribution, which allows for optimal parameter estimation.
- **Algorithm Performance:** Many popular algorithms, such as **Gaussian Naive Bayes** and **Logistic Regression**, perform more efficiently or yield more reliable results when features are normally distributed.
- **Feature Scaling:** Standardizing data (Z-score normalization) transforms features into a normal distribution with $\mu = 0$ and $\sigma = 1$, which helps gradient-based optimization algorithms converge faster.

5. What is skewness in distributions and which distributions exhibit it?

Skewness refers to the degree of asymmetry in a probability distribution. If a distribution is not symmetric (like the Normal distribution), it is considered skewed.

- **Positive (Right) Skew:** The distribution has a long tail extending toward the higher values on the right. This often happens when there is a lower limit (like 0) but no upper limit.
- **Negative (Left) Skew:** The distribution has a long tail extending toward the lower values on the left. This occurs when data is capped at a high maximum.

Distributions exhibiting skewness:

1. **Poisson Distribution:** Highly right-skewed when λ is small.
2. **Exponential Distribution:** Used to model the time between events; it is inherently right-skewed.
3. **Log-Normal Distribution:** Common in wealth and income data, where a few individuals earn significantly more than the average, creating a long right tail.
4. **Beta Distribution:** Can be left-skewed, right-skewed, or symmetric depending on its shape parameters.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm, poisson, binom
from google.colab import drive

drive.mount('/content/drive')

file_path = '/content/drive/MyDrive/camera_dataset.csv'
df = pd.read_csv(file_path)

df = df.dropna(subset=['Price'])

print("Dataset Loaded Successfully!")
print(df.head())

mu_price = df['Price'].mean()
std_price = df['Price'].std()

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
sns.histplot(df['Price'], kde=True, stat="density", color='skyblue')
x = np.linspace(df['Price'].min(), df['Price'].max(), 100)
plt.plot(x, norm.pdf(x, mu_price, std_price), color='red', lw=2, label='Norm')
plt.title(f"Normal Dist: Price\n $\mu = \{mu\_price:.2f\}$ ,  $\sigma = \{std\_price:.2f\}$ ")
plt.legend()

releases_per_year = df.groupby('Release date').size()
lam = releases_per_year.mean()

plt.subplot(1, 3, 2)
plt.bar(releases_per_year.index, releases_per_year.values, color='green', align='center')
plt.title(f"Poisson Analysis: Releases/Year\nAvg ( $\lambda$ ) =  $\{lam:.2f\}$ ")
```

```
plt.xlabel('Year')
plt.ylabel('Count of Models')

n = 20
p_success = (df['Price'] > 500).sum() / len(df)

plt.subplot(1, 3, 3)
x_binom = np.arange(0, n + 1)
plt.stem(x_binom, binom.pmf(x_binom, n, p_success))
plt.title(f"Binomial Dist (n=20, p={p_success:.2f})\nSuccess: Price > $500")
plt.xlabel('Number of Successes')
plt.ylabel('Probability')

plt.tight_layout()
plt.show()
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call Dataset Loaded Successfully!

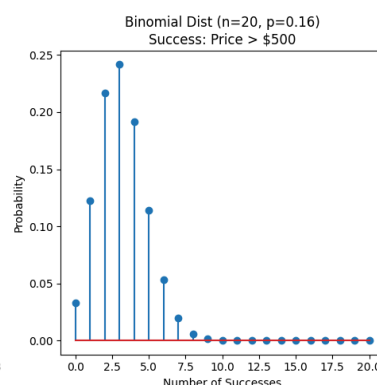
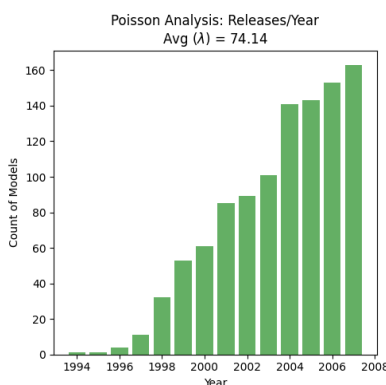
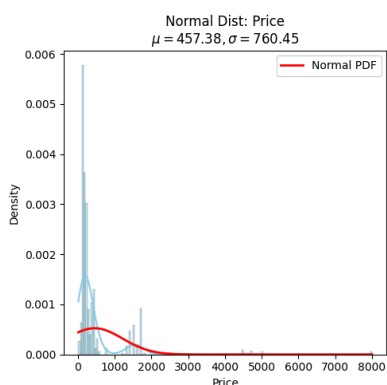
	Model	Release date	Max resolution	Low resolution \
0	Agfa ePhoto 1280	1997	1024.0	640.0
1	Agfa ePhoto 1680	1998	1280.0	640.0
2	Agfa ePhoto CL18	2000	640.0	0.0
3	Agfa ePhoto CL30	1999	1152.0	640.0
4	Agfa ePhoto CL30 Clik!	1999	1152.0	640.0

	Effective pixels	Zoom wide (W)	Zoom tele (T)	Normal focus range \
0	0.0	38.0	114.0	70.0
1	1.0	38.0	114.0	50.0
2	0.0	45.0	45.0	0.0
3	0.0	35.0	35.0	0.0
4	0.0	43.0	43.0	50.0

	Macro focus range	Storage included	Weight (inc. batteries)	Dimensions
0	40.0	4.0	420.0	95.0
1	0.0	4.0	420.0	158.0
2	0.0	2.0	0.0	0.0
3	0.0	4.0	0.0	0.0
4	0.0	40.0	300.0	128.0

Price

0	179.0
1	179.0
2	179.0
3	269.0
4	1299.0



✓ CONCLUSION

Through this practical, the implementation and comparative analysis of **Binomial**, **Poisson**, and **Normal** distributions were successfully performed using Python's `scipy.stats` and `matplotlib` libraries. The following key insights were observed:

- **Discrete vs. Continuous Modeling:** We demonstrated that while the Binomial and Poisson distributions are effective for modeling discrete counts (like coin

flips or event frequencies), the Normal distribution is essential for continuous data that clusters around a central mean.

- **Parameter Sensitivity:** By adjusting parameters such as n , p , and λ , we observed how the shape of a distribution shifts. Specifically, we noted that as the number of trials or the rate increases, discrete distributions begin to approximate the bell-shaped curve of a Normal distribution, illustrating the **Central Limit Theorem**.
- **Statistical Analysis:** The practical highlighted the concept of **skewness**, showing how asymmetry in data affects the distribution's tail. Understanding these shapes is vital for selecting the correct machine learning models and performing accurate hypothesis testing.
- **Computational Implementation:** The use of Python allowed for the efficient generation of large datasets and the visualization of complex Probability Mass Functions (PMF) and Probability Density Functions (PDF), reinforcing the mathematical theory with practical application.

Conclusion

This practical successfully demonstrated the application of **Normal Poisson and**